Estructuras de Datos y Algoritmos Fundamentales

Escuela de Ingeniería y Ciencias

Nombre del Estudiante: _____
Matrícula: _____ **Fecha:** _____

Ejercicio 1: Análisis de Complejidad (25 pts)

Dada una función recursiva, se ha modelado su tiempo de ejecución $T(n)$ con la siguiente ecuación de recurrencia:

$$T(n) = \begin{cases} 1 & \text{si } n = 1 \\ T(n/2) + 1 & \text{si } n > 1 \end{cases}$$

Resuelva la recurrencia utilizando el **método de sustitución** para encontrar la fórmula cerrada de $T(n)$. Posteriormente, confirme utilizando el teorema maestro.

Ejercicio 2: Posición de Inserción (25 pts)

Dado un arreglo de números enteros ordenado de manera ascendente y un valor objetivo `target`, escriba una función en **C++** que determine la posición en la que se encuentra dicho valor (indique su **complejidad temporal y espacial**).

Si el valor no se encuentra en el arreglo, la función deberá regresar el índice en el que debería insertarse para mantener el arreglo ordenado.

Por ejemplo:

$$A = [1, 3, 5, 6]$$

- Si `target = 5`, la función debe regresar `2`.
- Si `target = 2`, la función debe regresar `1`.

La solución deberá utilizar tener una complejidad temporal máxima de $\boldsymbol{O(\log n)}$.

Ejercicio 3: Algoritmos de Ordenamiento (25 pts)

Considere el siguiente arreglo de números enteros positivos que contiene las edades de los estudiantes:

$$\boldsymbol{A = [18, 19, 20, 19, 19, 21, 18, 19, 20, 21]}$$

Implemente una función en C++ para ordenar las edades en tiempo máximo $\boldsymbol{O(n)}$. Asuma que la edad máxima de los estudiantes es 25 (indique su **complejidad temporal y espacial**).

Ejercicio 4: Reverse Linked List (25 pts)

Considere la siguiente definición de un nodo para una lista simplemente enlazada en C++:

```
struct ListNode {
    int val;
    ListNode *next;

    ListNode(int x) : val(x), next(NULL) {}
};
```

Implemente una función en **C++** que invierta una lista simplemente enlazada. (indique su **complejidad temporal y espacial**).

Por ejemplo, dada la lista:

$$\mathbf{1} \rightarrow \mathbf{2} \rightarrow \mathbf{3} \rightarrow \mathbf{4} \rightarrow \texttt{NULL}$$

la función deberá modificar sus enlaces para obtener:

$$\mathbf{4} \rightarrow \mathbf{3} \rightarrow \mathbf{2} \rightarrow \mathbf{1} \rightarrow \texttt{NULL}$$

La solución no deberá crear una segunda lista con copias de los nodos. Deberá modificar los apuntadores de la lista existente.

Ejercicio 5: Brincos Recursivos (Extra 10 pts)

Una persona desea subir una escalera de $\boldsymbol{n}$ escalones. En cada movimiento puede avanzar exactamente: **1 escalón**, **2 escalones**.
Escriba una función **recursiva** en C++ que calcule el número total de formas diferentes en las que la persona puede llegar exactamente al escalón $\boldsymbol{n}$.

Por ejemplo:

- Para $\boldsymbol{n = 1}$ existe **1** forma.
- Para $\boldsymbol{n = 2}$ existen **2** formas: $\boldsymbol{(1 + 1)}$ y $\boldsymbol{(2)}$.
- Para $\boldsymbol{n = 3}$ existen **3** formas: $\boldsymbol{(1 + 1 + 1)}$, $\boldsymbol{(1 + 2)}$ y $\boldsymbol{(2 + 1)}$.